Forklift Controller
User Manual v2.0.0



WSM GAME STUDIO

Doing all the hard work for you!

2026

SUMMARY

1. Intro **3**
1.1. RP Compatibility (URP Upgrade) 3
2. How to Use This Asset **3**
2.1. What can it do? (Quick Start Guide) 4
2.2. How it Works 7
3. Forklift Modules **8**
3.1. Forklift Input 8
3.1.1. Forklift New Input System 8
3.1.2. Forklift Legacy Input System 9
3.1.3. Mapping Custom Forklift Events 10
3.2. Forklift Controller 11
4. Vehicle Modules **12**
4.1. Vehicle Input 12
4.1.1. Vehicle New Input System 13
4.1.2. Vehicle Legacy Input System 14
4.1.3. Mapping Custom Vehicle Events 15
4.2. Vehicle Controller 16
4.2.1. Engine Settings 16
4.2.1.1. Supported Drivetrain Types 16
4.2.2. Speed Settings 17
4.2.3. Gearbox Settings 17
4.2.4. Physics Settings 17
4.2.5. Steering Settings 18
4.2.5.1. Supported Steering Modes 18
4.2.5.2. Articulated Steering Vehicles 18
4.2.6. Wheel Settings 19
4.2.7. Vehicle Parts 20
4.2.8. Vehicle SFX 20
4.2.9. Vehicle Lights 21
4.2.9.1. Emissive Lights (Glow Mesh) 21
4.2.9.2. Legacy Light System 22
4.2.9.3. Shared Light Settings 22
4.2.10. Vehicle UI 23
5. Vehicle Camera Modules **23**
5.1. Camera Player Input 24
5.1.1. Camera New Input System 24

5.1.2. Camera Legacy Input System 25
5.2. Camera Component 27
5.2.1. Camera Mode 27
5.2.2. Target Vehicle & Target Settings 28
5.2.3. Supported Camera Types 29
5.2.4. Camera Collision 30
6. Additional Info **30**
6.1. Customization 30
6.2. Forklift Carrying Weight Capacity 30
6.3. Rear Steering Vehicle Guidelines 31
7. License **31**
8. Contact Info & Support **31**

1. Intro

Thank you for purchasing the Forklift Controller asset!

This package contains all that you need to include a simple and functional forklift to your game. It's really simple to use and customize.

For more information on related assets, please visit: https://wsmgamestudio.com/

Obs: By owning this pack, you're eligible for a **44% discount** if you decide to upgrade to the Heavy Machinery Collection.

Enjoy the asset? Please leave a review to support my work.

1.1. RP Compatibility (URP Upgrade)

To guarantee compatibility with all Render Pipelines, the asset is distributed as a built-in render pipeline project. If all materials are pink after downloading the asset, this means you've downloaded the asset on a URP or HDRP project.

To upgrade all materials, please install the corresponding upgrade package located in the "Forklift Controller/SRP Support Packages" folder.



Figure: SRP Upgrade Packages

Obs: If any materials are still pink after installing the upgrade package, go to "*Window* > *Rendering* > *Render Pipeline Converter*" to convert any remaining materials.

2. How to Use This Asset

In this section you will learn how this asset works and how to use it based on the sample prefabs.

2.1. What can it do? (Quick Start Guide)

This asset contains ready-to-use forklift prefabs which you can drive and use to lift and move cargo around your game. Modular pallet racks and wooden pallet prefabs are also included.

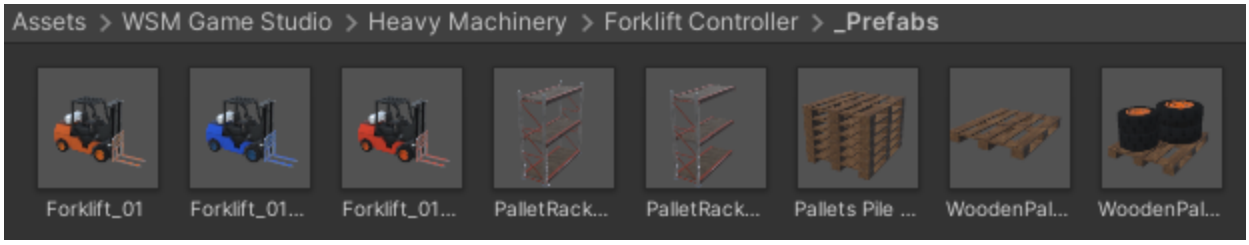


Figure: Sample Prefabs

Most real life forklifts use **rear wheel steering** for greater maneuverability while handling cargo in tight spaces. However, **front wheel steering** forklifts are also supported.



Figure: Front Wheels Steering x Rear Wheels Steering

To see these prefabs in action, please take a look at the demo scenes included on the package.

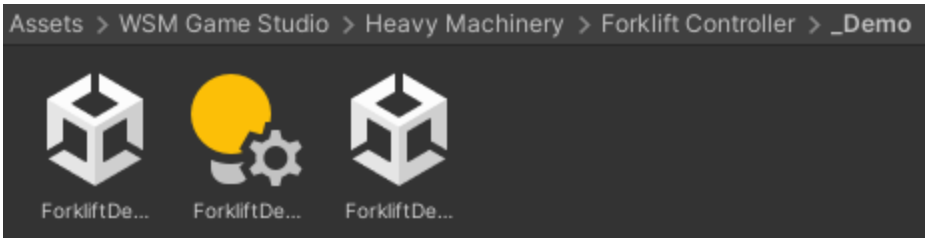


Figure: Demo Scenes

Hint: By pressing the Tab key on the demo scenes you can show or hide the current forklift input keys. This can be useful for testing the forklift for the first time.

4 distinct forklift fork movements are supported.

- Vertical Fork Movement
- Horizontal Fork Movement
- Fork Width Adjustment
- Mast Tilting

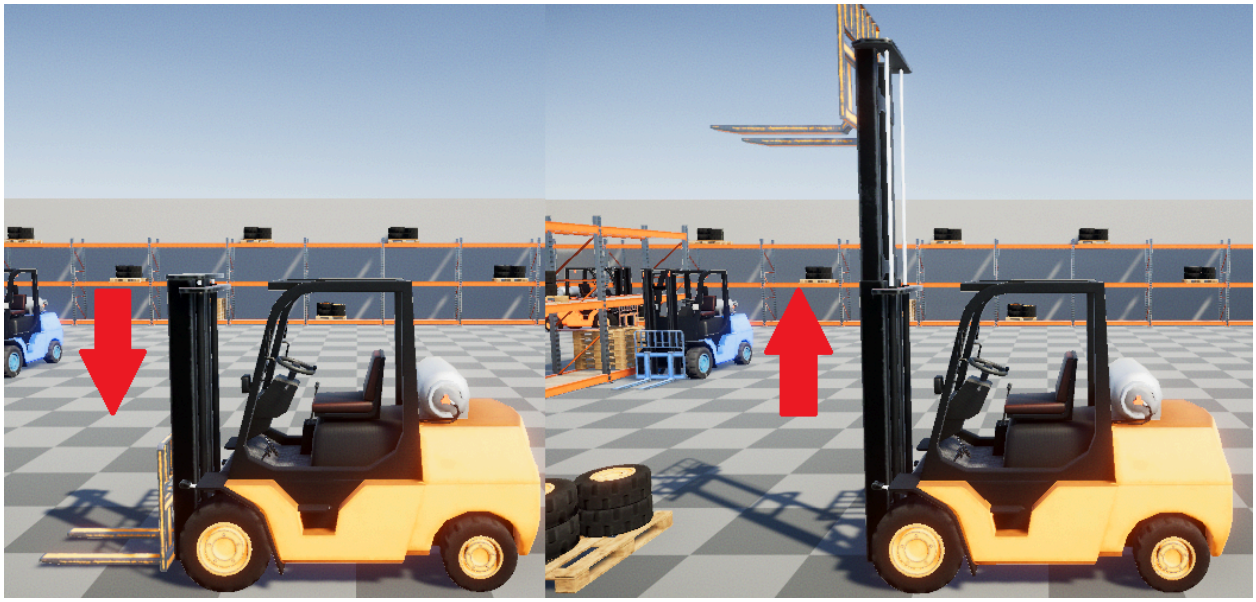


Figure: Vertical Fork Movement

Vertical movement is used to lift and lower cargo, by default it can reach approximately up to 3.45 meters high.



Figure: Horizontal Fork Movement

Horizontal movement lets you precisely position cargo without steering. Forks are centered by default and have a horizontal reach of 42cm (21cm left and right).

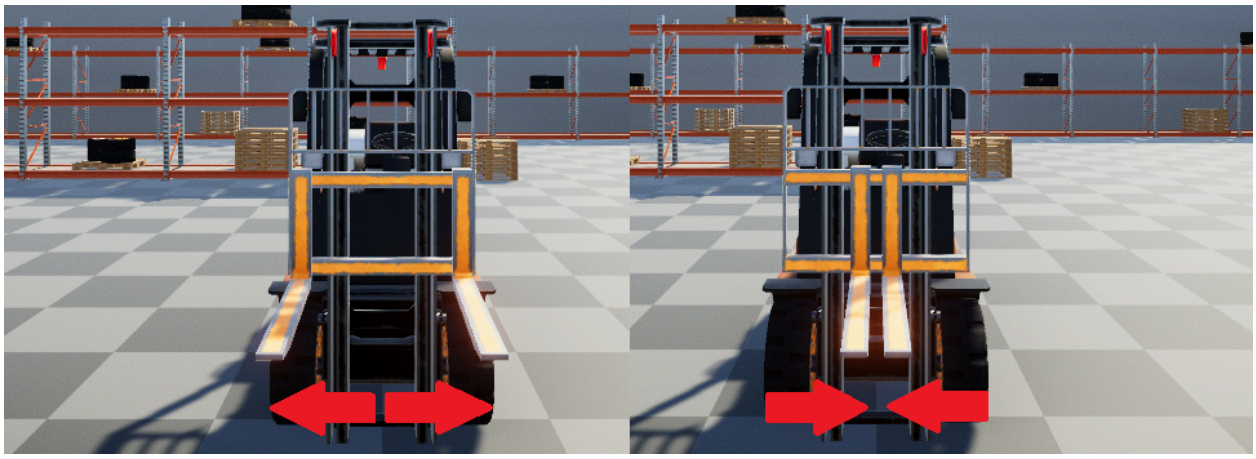


Figure: Fork Width Adjustment

The fork width is fully adjustable, allowing for handling different pallet sizes with optimal spacing for increased stability.

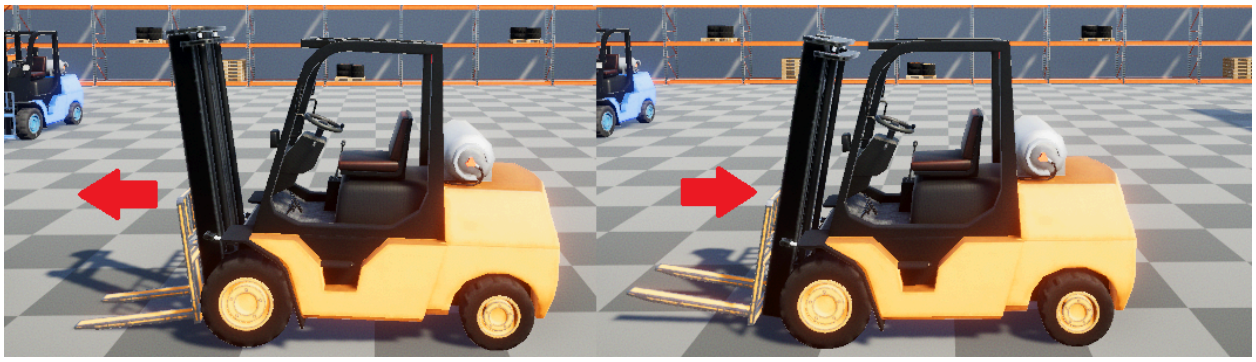


Figure: Mast Tilting

Mast tilting helps prevent loose cargo from falling. For example, tilting the mast backwards while carrying a stack of wooden pallets increases the stack stability and avoids accidents.



Figure: Mast Tilting Sample

2.2. How it Works

This asset was designed with modularity and easy customization in mind. Therefore, the Component Game Design Pattern was implemented to keep player input, forklift behavior and vehicle behavior as decoupled as possible.

Each forklift is composed of four main modules:

- Forklift Player Input
- Vehicle Player Input
- Forklift Controller
- Vehicle Controller

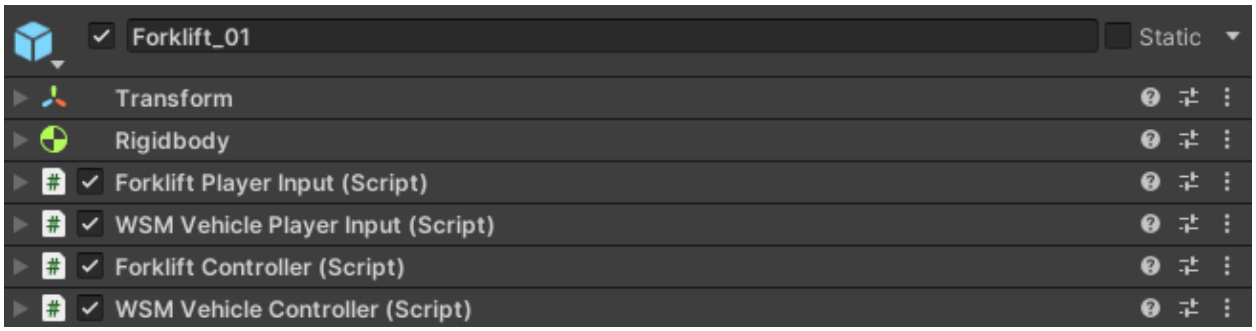


Figure: Forklift Components

The forklift and vehicle input modules are responsible to gather and feed player input into the corresponding controller modules. For more information about these modules and how to customize input, please take a look at the Forklift Input and Vehicle Input sections.

The Forklift Controller component is responsible for handling all forklift specific behavior. Which means, handling the fork movements and SFX. For more information, please take a look at the Forklift Controller section.

The Vehicle Controller component is responsible for handling all vehicle specific behavior (Ex: acceleration, steering, vehicle physics, headlights, etc.). For more information, please take a look at the Vehicle Controller section.

This modular approach allows customization by replacing any module with third-party or custom modules. For example, a client that already owns a custom or third-party vehicle controller may wish to replace the default vehicle modules and still control the forklift's mechanical parts using the Forklift Controller component.

3. Forklift Modules

This section covers the forklift related modules and settings.

3.1. Forklift Input

This component is responsible for handling player input for all forklift specific behaviors. Starting with v2.0, you may choose between the New Input System or the asset's Legacy Input System.

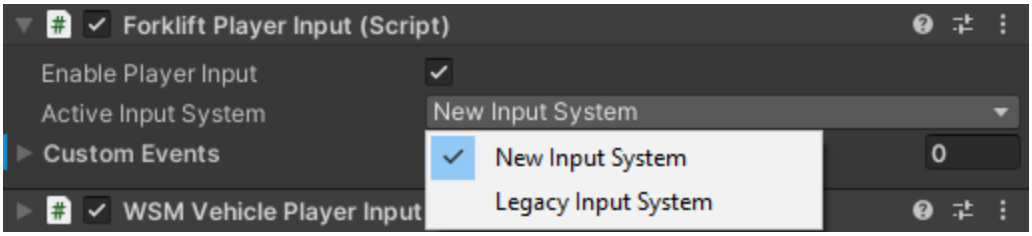


Figure: Active Forklift Input System

3.1.1. Forklift New Input System

The **new input system** leverages the power of the Input System Package to support a wide range of target platforms and controller devices (supported input devices).

The forklift related input mappings are located on the "*ForkliftInputMapping.inputactions*" file.

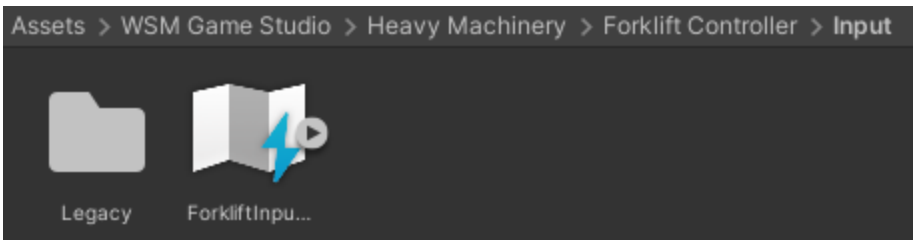


Figure: Forklift Input Action Asset

By opening this file, you may expand and customize the input mappings as you see fit.



Figure: Forklift Input Mappings

Sample mappings include control schemes for PC (**Keyboard**), Xbox and Playstation (**Gamepad**), and generic controllers (**USB Game Pad**).

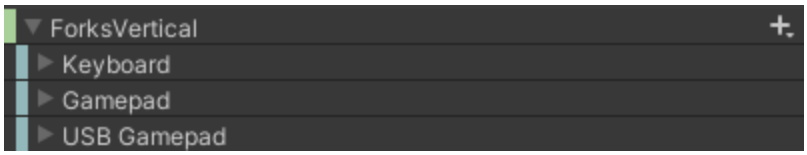


Figure: Input Mappings Sample

For example, on a **Xbox Controller**, forks can be lifted by holding **LB** and **D-pad Up.**
(**L1** and **D-pad Up** on a **Playstation Controller**).

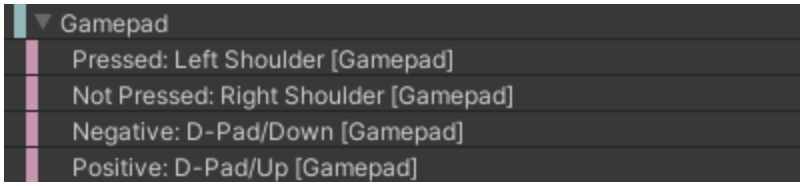


Figure: Console Input Sample (Xbox, Playstation)

3.1.2. Forklift Legacy Input System

The **legacy input system** is an improved version of Unity's old Input Manager system (input mappings are stored on scriptable objects instead of the Project Settings).

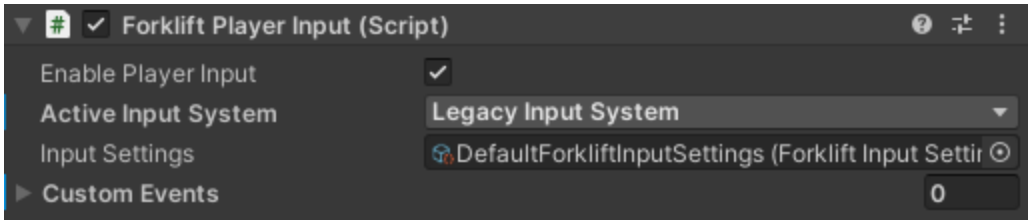


Figure: Forklift Player Input

When the **legacy input system** is selected, the forklift input mappings are stored on a *Forklift Input Settings* file.

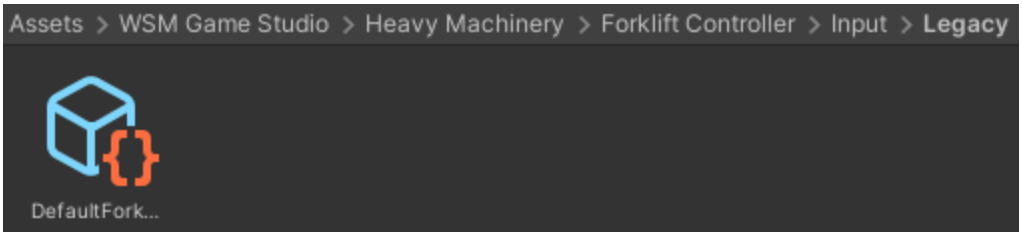


Figure: Forklift Input Settings Location

By selecting this file, you can see and customize the forklift input mappings.

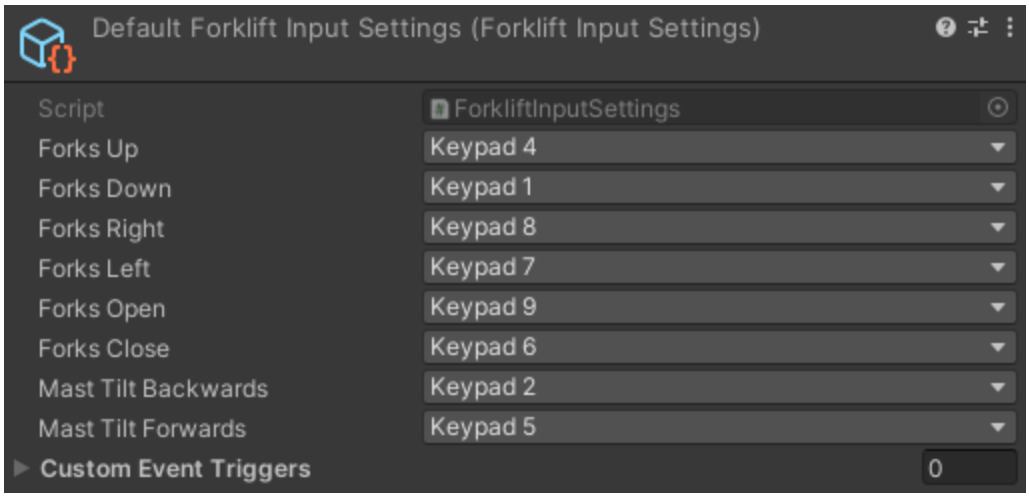


Figure: Forklift Input Settings

You can create custom input settings by right-clicking on your project and using Unity's Asset Creation menu: *Create > WSM Game Studio > Heavy Machinery > Legacy Input System > Forklift Input Settings*.

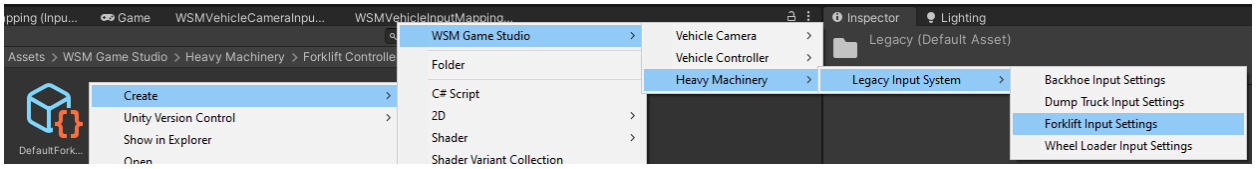


Figure: Creating a new Forklift Input Settings

You can create as many input settings as you wish, this can be very useful for testing different input mappings for your game.

> **Obs:** The legacy input system supports keyboard input only. For extended controller support, please use the new input system instead.

3.1.3. Mapping Custom Forklift Events

If the sample forklift input mappings aren't enough for your project, both new and legacy input systems support input mappings for custom events. You can reference any public method from

custom or third-party scripts in the **Custom Events** array, facilitating customization and integration with external solutions. Just make sure the events and input mappings are set up in the desired order.

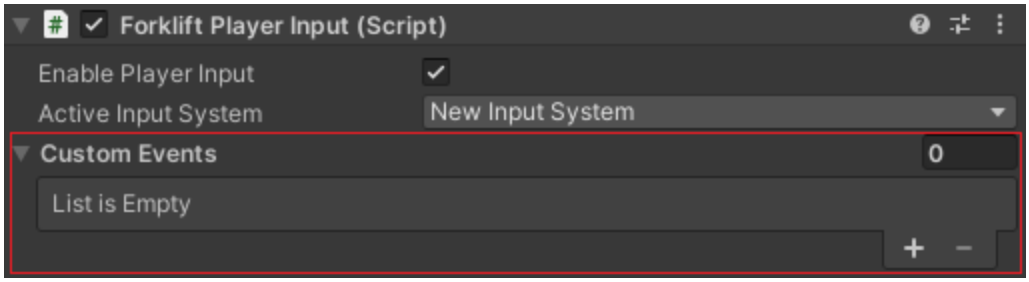


Figure: Custom Forklift Events

3.2. Forklift Controller

This component is responsible for controlling the fork's mechanical movements.

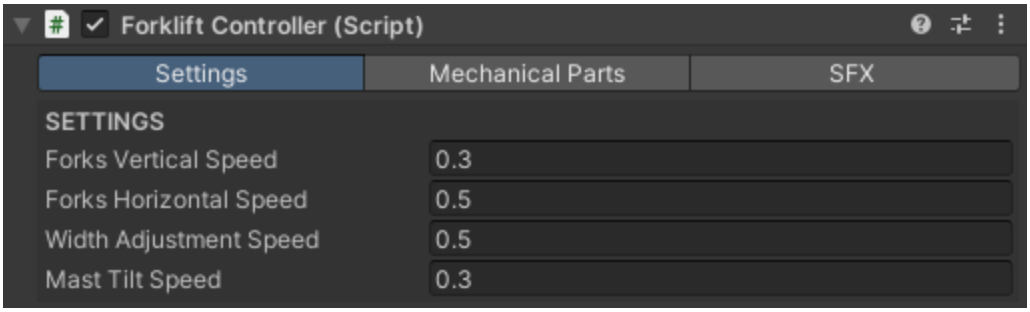


Figure: Forklift Controller

At the Settings tab, you can set up the forklift mechanical parts movement speed.

> **Note:** The forklift mechanical parts only move when the vehicle engine is **on**.

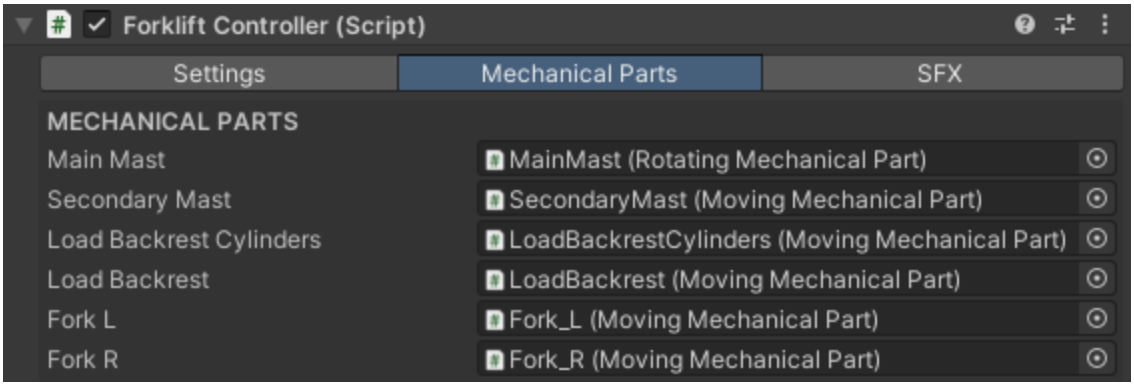


Figure: Forklift Mechanical Parts

The Mechanical Parts tab holds the references to all forklift mechanical parts.



Figure: Forklift Levers

The levers responsible for moving each part are also referenced under this tab.

Note: These references are used to simulate the operator moving the levers at the cabin.

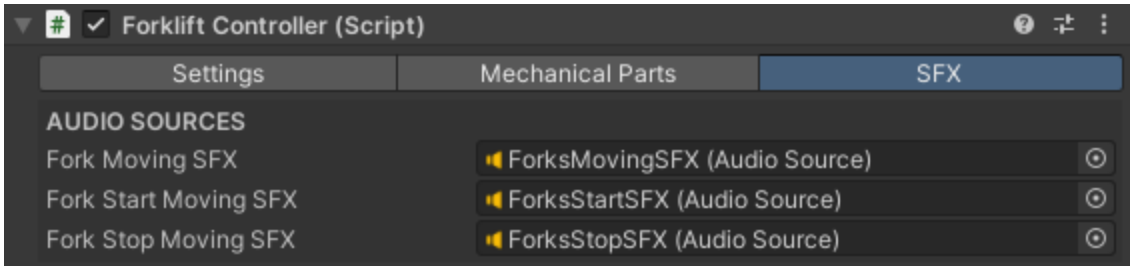


Figure: Forklift SFX

The SFX tab holds references to all the fork movements related Audio Sources.

4. Vehicle Modules

This section covers the vehicle related modules and settings.

4.1. Vehicle Input

This component is responsible for handling player input for all vehicle specific behaviors. Starting with v2.0, you may now choose between the New Input System or the asset's Legacy Input System.

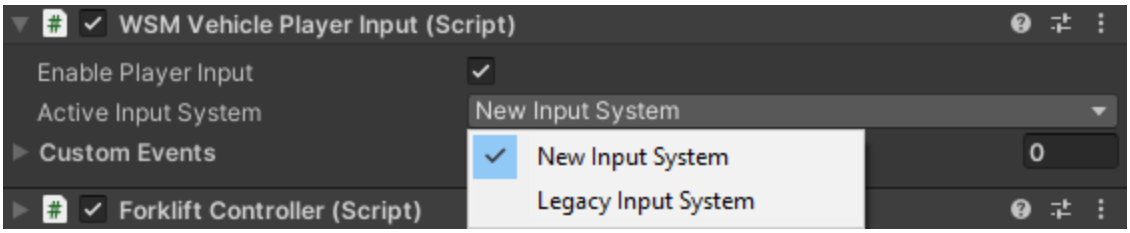


Figure: Active Vehicle Input System

4.1.1. Vehicle New Input System

The **new input system** leverages the power of the Input System Package to support a wide range of target platforms and controller devices (supported input devices).

The vehicle related input mappings are located on the "*WSMVehicleInputMapping.inputactions*" file.

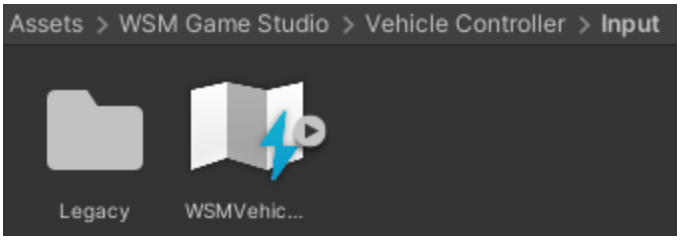


Figure: Vehicle Input Action Asset

By opening this file, you may expand and customize the input mappings as you see fit.

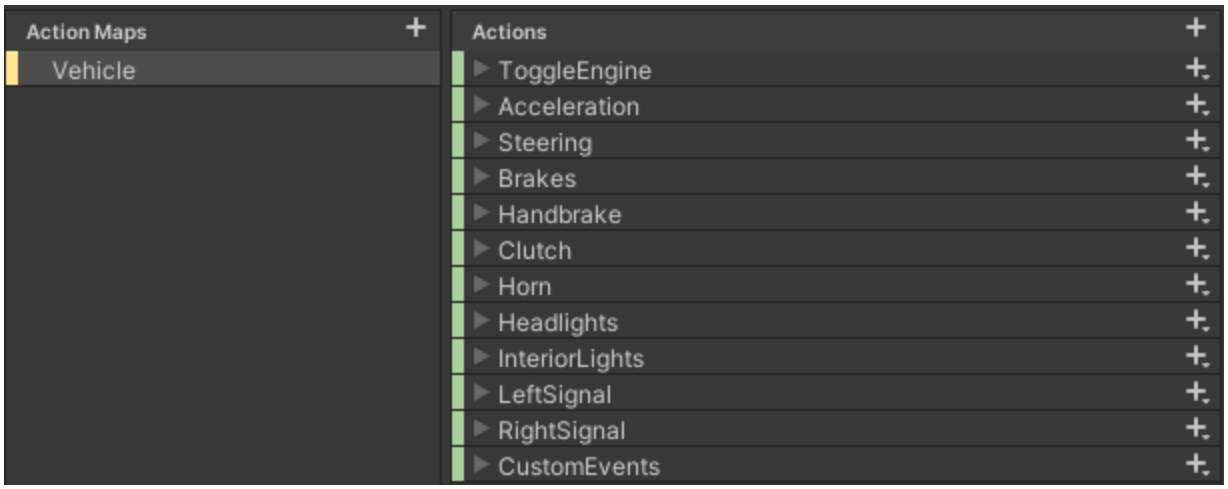


Figure: Vehicle Input Mappings

Sample mappings include control schemes for PC (**Keyboard**), Xbox and Playstation (**Gamepad**), and generic controllers (**USB Game Pad**).

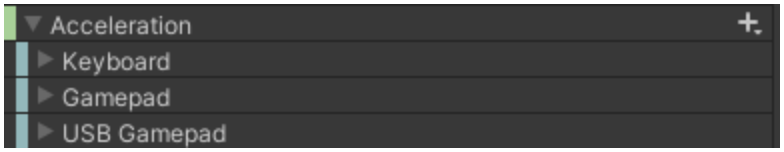


Figure: Input Mappings Sample

For example, on a **Xbox Controller**, acceleration and reverse are mapped to **RT** and **LT**.
(**R2** and **L2** on a **Playstation Controller**).

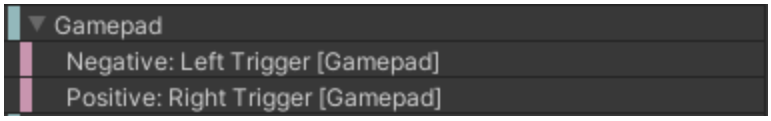


Figure: Console Input Sample (Xbox, Playstation)

4.1.2. Vehicle Legacy Input System

The **legacy input system** is an improved version of Unity's old Input Manager system (input mappings are stored on scriptable objects instead of the Project Settings).

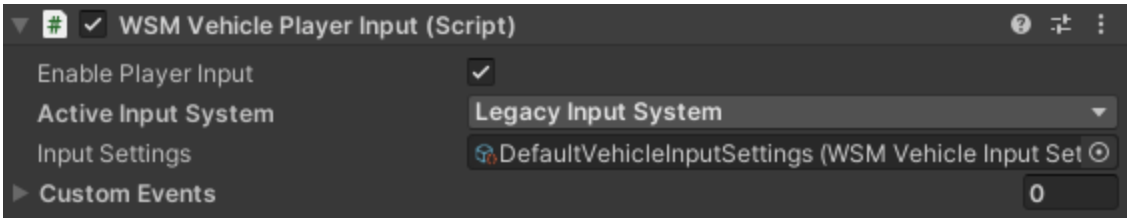


Figure: Vehicle Player Input

When the **legacy input system** is selected, the vehicle input mappings are stored on a Vehicle Input Settings file.

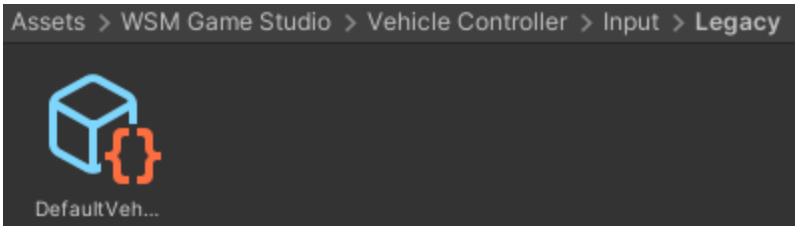


Figure: Vehicle Player Input Location

By selecting this file, you can see and customize the input keys used to control your vehicle.

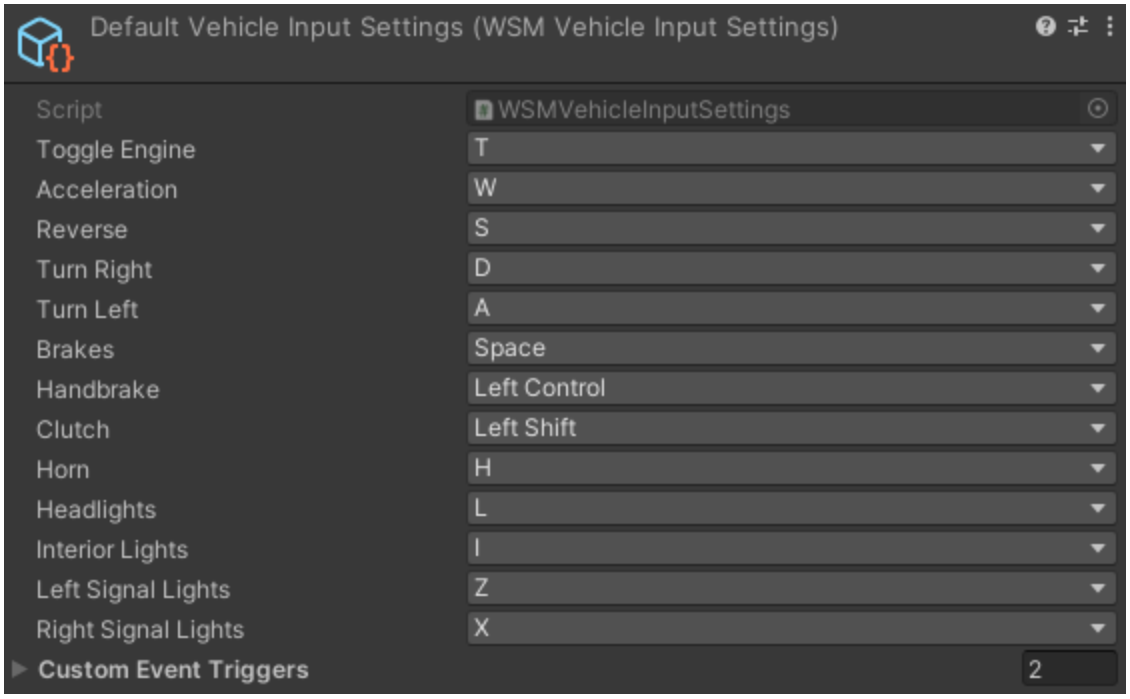


Figure: Vehicle Input Settings

You can also create new custom input settings by right-clicking on your project and using the Unity Asset Creation menu: "*Create* > *WSM Game Studio* > *Vehicle Controller* > *Legacy Input System* > *Vehicle Input Settings*".

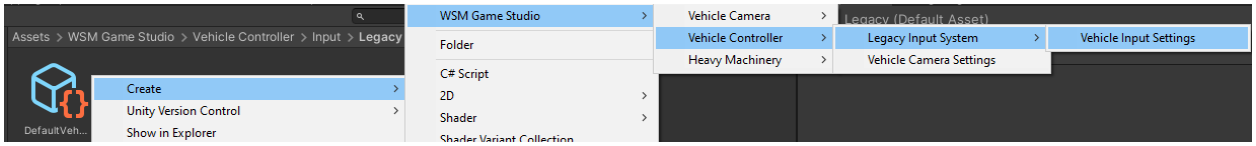


Figure: Creating a new Vehicle Input Settings

4.1.3. Mapping Custom Vehicle Events

If the sample vehicle input mappings aren't enough for your project, both new and legacy input systems support input mappings for custom events. You can reference any public method from custom or third-party scripts in the **Custom Events** array, facilitating customization and integration with external solutions. Just make sure the events and input mappings are set up in the desired order.

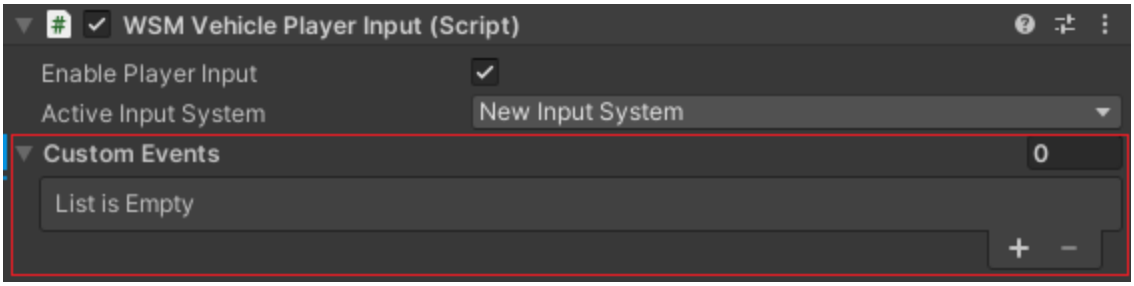


Figure: Custom Vehicle Events

4.2. Vehicle Controller

This component is responsible for controlling vehicle-related features and vehicle physics. Most vehicle settings located here are very straightforward and use the same nomenclature as their corresponding real-world counterparts. The most specific settings will be covered in this section.

> **Note:** If you have any questions about any specific setting that was not covered in this section, feel free to email me about it (See Contact Info & Support section)

4.2.1. Engine Settings



Figure: Vehicle Engine Settings

In the **engine settings** section, you can turn the engine on/off and select the vehicle's drivetrain type.

4.2.1.1. Supported Drivetrain Types

- Front-wheel Drive (FWD)
- Rear-wheel drive (RWD)
- All-wheel drive (AWD)

4.2.2. Speed Settings

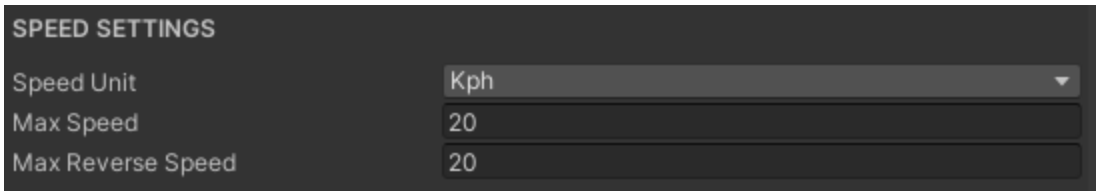


Figure: Vehicle Speed Settings

In the **speed settings** section, you can define the vehicle's speed limits and desired speed unit (Kph or Mph).

4.2.3. Gearbox Settings



Figure: Vehicle Gearbox Settings

In the **gearbox settings** section, you can set the number of gears and traction related settings.

4.2.4. Physics Settings

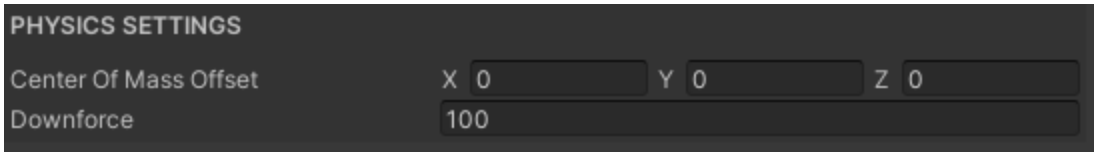


Figure: Vehicle Physics Settings

In the **physics settings** section, you can customize the vehicle center of mass and downforce.

4.2.5. Steering Settings

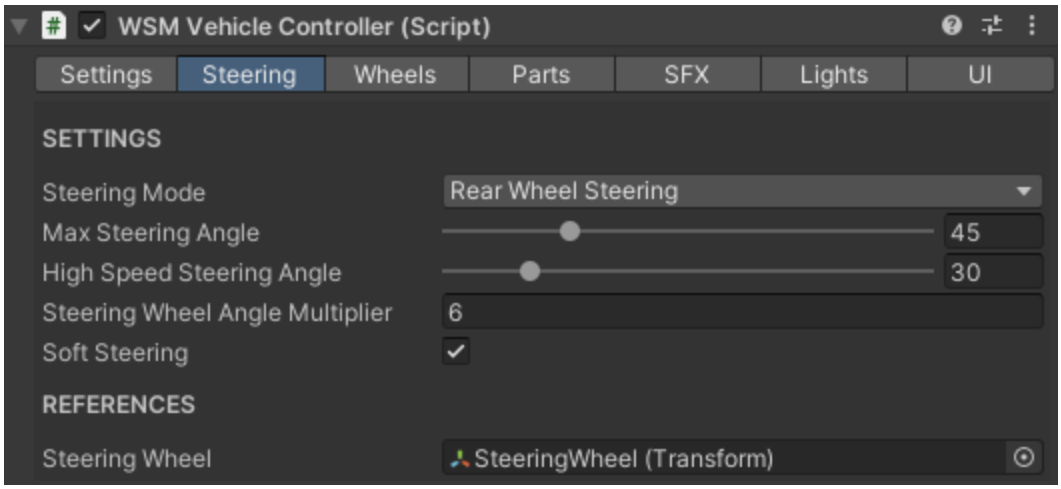


Figure: Vehicle Steering Settings

In the **steering section**, you can select the steering mode and other steering related properties.

4.2.5.1. Supported Steering Modes

- Front Wheel Steering
- Rear Wheel Steering
- Articulated Steering

4.2.5.2. Articulated Steering Vehicles

This section covers Articulated vehicles steering settings.

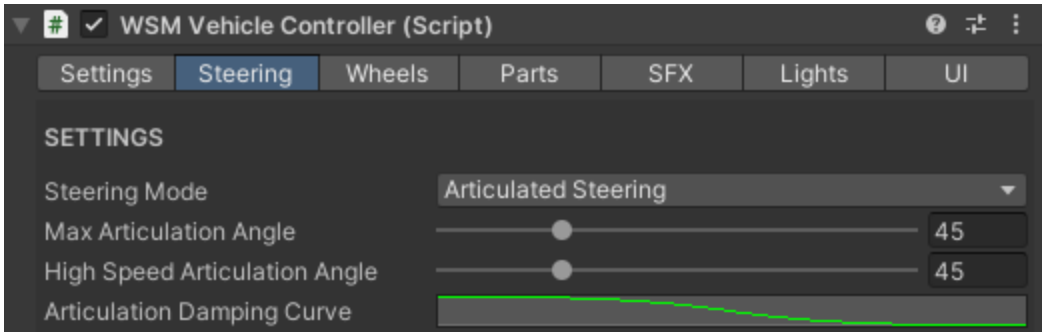


Figure: Articulated Steering Settings

The **max articulation angle** defines the vehicle maximum turning radius. The **high speed articulation angle** may also be used to simulate high-speed steering resistance.
Since Unity Joints Components are non-deterministic, the vehicle articulation physics are simulated using a deterministic algorithm instead. This ensures vehicle stability and performance at the cost of reduced articulation realism (a common trade-off in game

development). Long story short, the **articulation damping curve** mathematical function simulates the unique articulation physics involved in pulling and pushing connected vehicle segments.

Given the differences between pulling and pushing forces, the front and rear sections of the vehicle must be treated and referenced independently, as shown in the following sample.

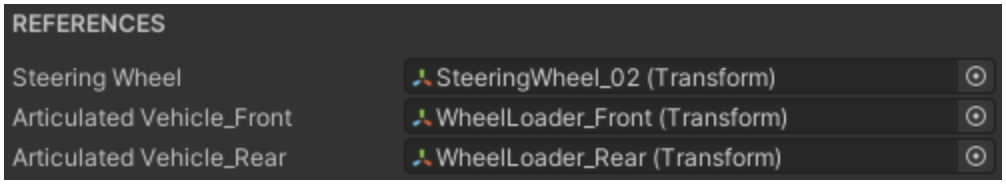


Figure: Front & Rear Articulation References

Note: Most forklifts are rear wheel steering. Check out the Wheel Loader Controller asset for an articulated steering vehicle sample.

4.2.6. Wheel Settings



Figure: Vehicle Wheels

The **wheels section** holds references to all wheel colliders and **meshes**. Front and rear references are separated, giving you full control over traction and steering.

Note: Traction is defined by the ***Drivetrain Type*** property and steering is defined by the ***Steering Mode*** property (steering tab)

If your vehicle has more than four wheels, you can also use the Extra Wheels arrays to assign the additional wheel references.

4.2.7. Vehicle Parts

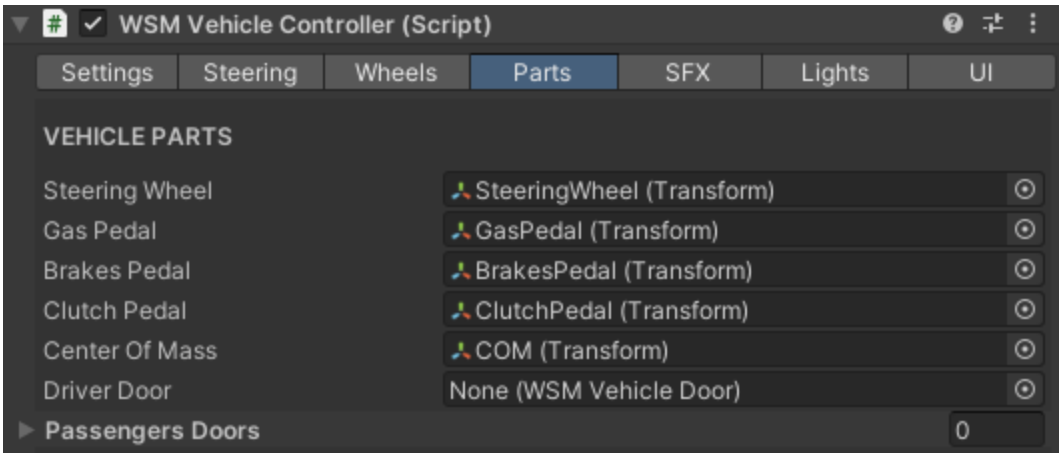


Figure: Vehicle Parts

The **parts section** holds references to all vehicle's moving parts. If your vehicle has an open cabin or if your game supports cabin interior view, this is where you can set a reference to your steering wheel and pedals.

The Center of Mass reference is used to visualize the vehicle center of mass position. Which can be adjusted by the ***Center of Mass Offset*** property (Physics settings).

4.2.8. Vehicle SFX

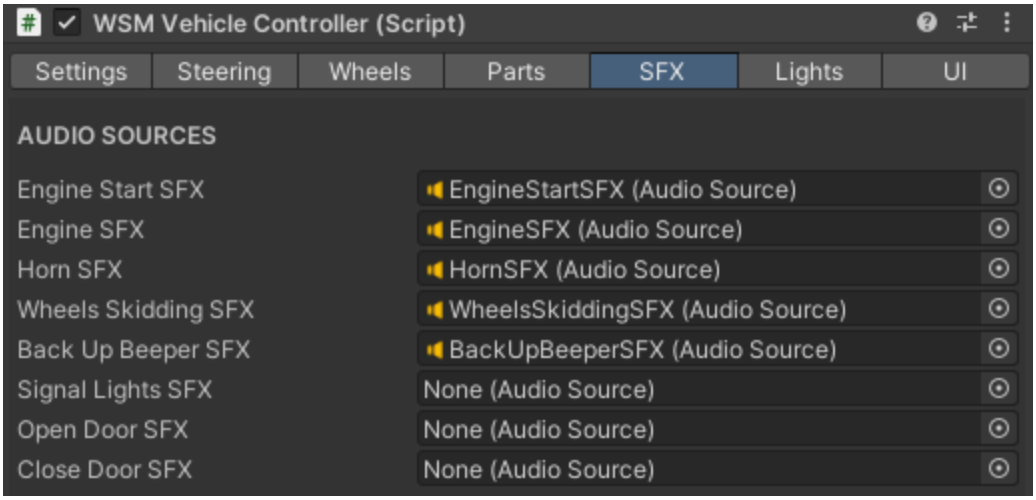


Figure: SFX References

The SFX tab holds references to the vehicle's Audio Sources.

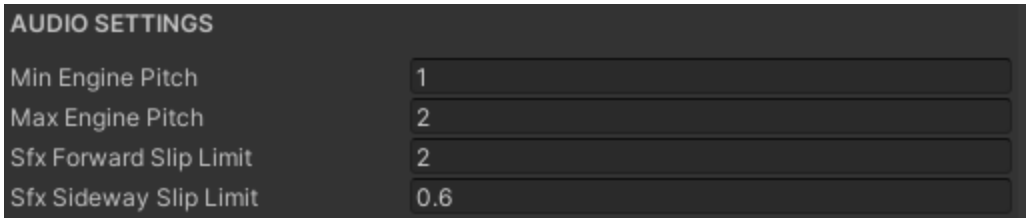


Figure: Audio Settings

Audio settings are also located here. The min and max engine pitch properties are used to simulate engine revving and gear switching sounds. The slip limit properties adjust when the wheel skidding SFX plays.

4.2.9. Vehicle Lights

Starting with v2.0, you may now choose between the Glow Mesh, for emissive lights support, or the asset's Legacy Light System.

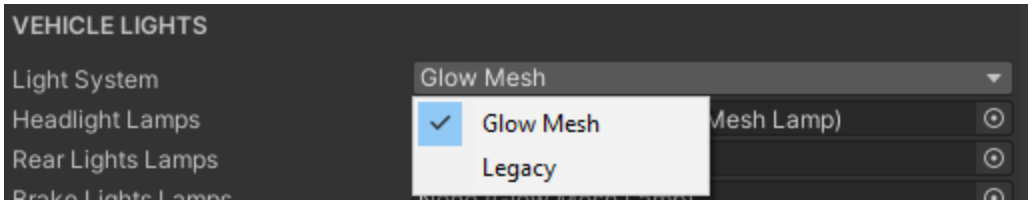


Figure: Active Lighting System

4.2.9.1. Emissive Lights (Glow Mesh)

When Glow Mesh is selected, vehicle lights are simulated by the **Glow Mesh Lamp** component, using a combination of emissive materials and light components to create realistic headlights.

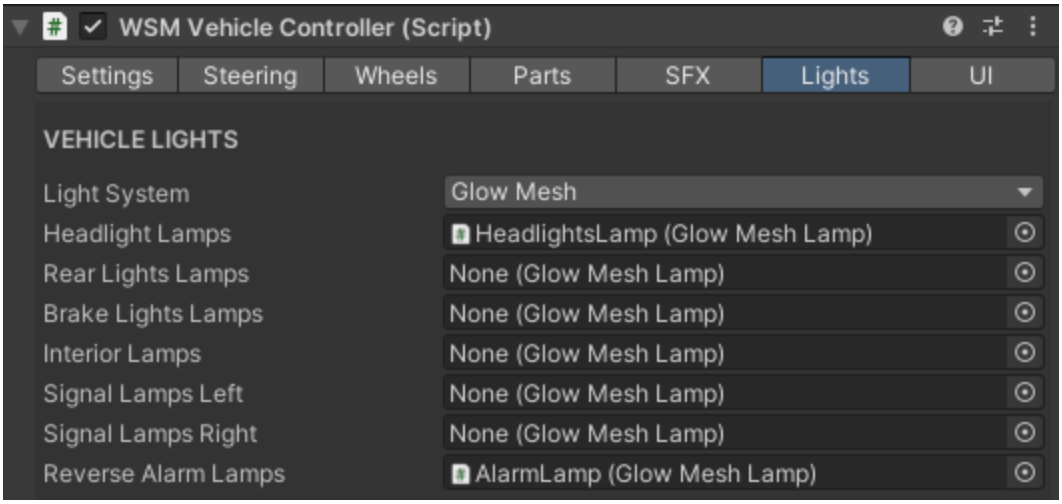


Figure: Glow Mesh Settings

Here is a sample of the glow mesh lamp component in action.



Figure: Emissive Headlights Sample

Note: This package includes a **minimal install** of the Glow Mesh asset (available on the Unity Asset Store).

4.2.9.2. Legacy Light System

The legacy light system is based exclusively on light components (no emissive lights support).

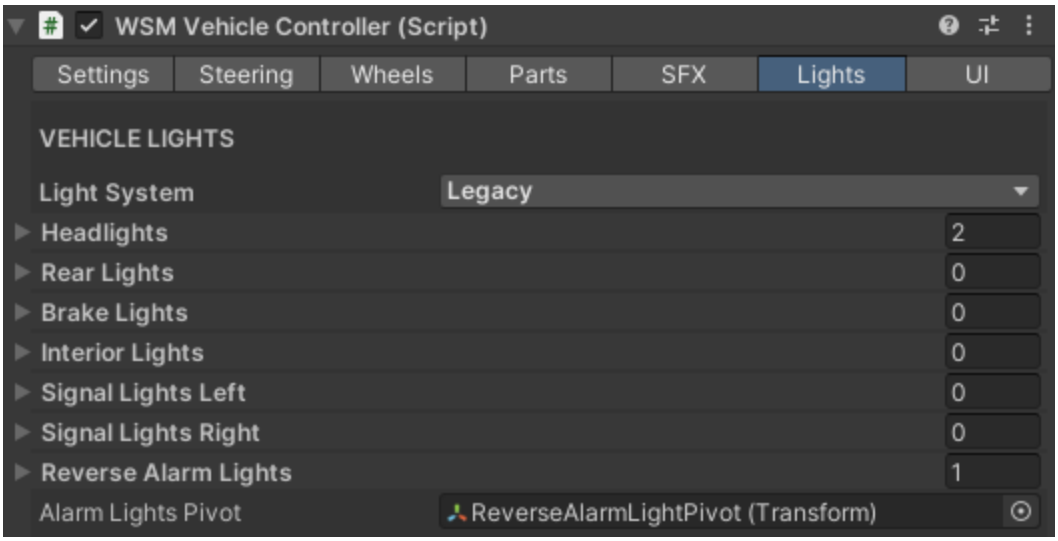


Figure: Legacy Light Settings

4.2.9.3. Shared Light Settings

The following settings are shared by both light systems.

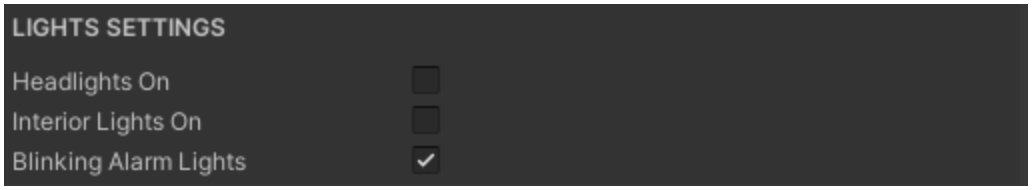


Figure: Lighting Settings

Note: Alarm lights rotation refers to the ***Reverse Alarm Lights***. Some trucks and other heavy vehicles usually have rotating lights to warn pedestrians, besides the backup beeper sound.

4.2.10. Vehicle UI

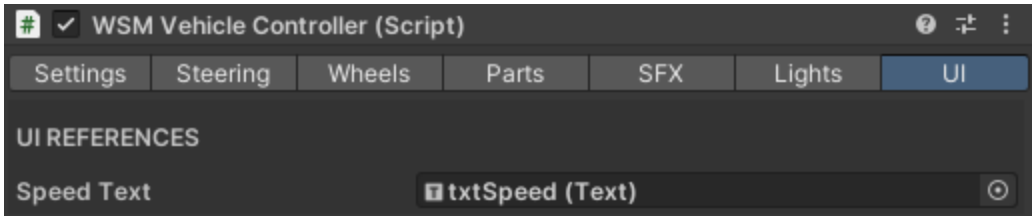


Figure: Vehicle UI

The UI tab holds a reference to the vehicle related UI. For now, it contains only an option for outputting the current vehicle speed.

Note: You can choose Kph and Mph by changing the ***speed unit*** property (Speed settings).

5. Vehicle Camera Modules

This asset includes a game-ready vehicle **camera prefab**, compatible with any type of vehicle.

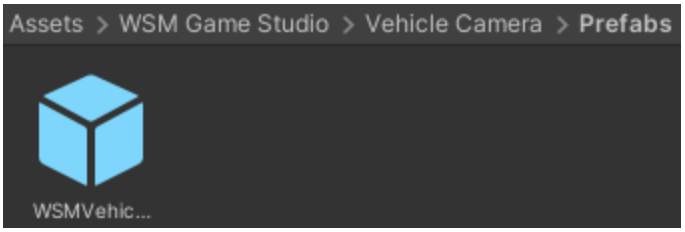


Figure: Vehicle Camera Prefab

The camera prefab is mainly composed of the following modules:

- Vehicle Camera Player Input
- Vehicle Camera Component

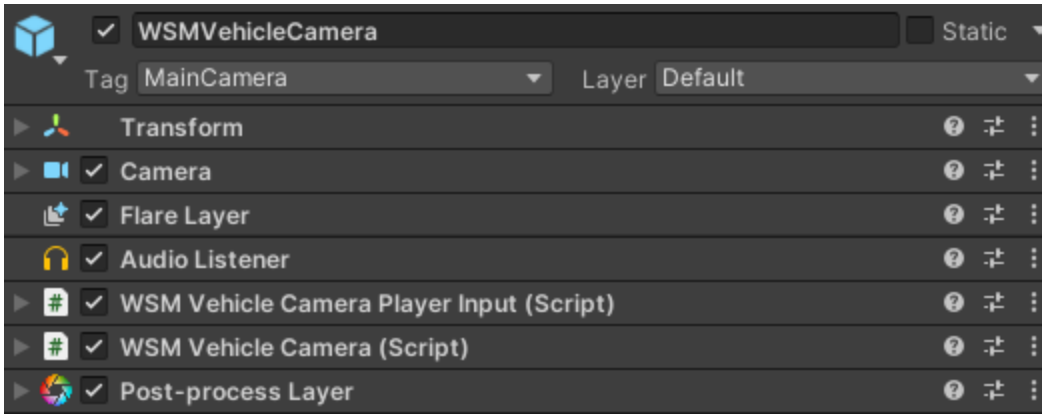


Figure: Vehicle Camera Components

Note: Starting with v2.0, the **vehicle camera** was completely detached from the vehicle modules, allowing the camera prefab to be used with any type of vehicle, including custom or third-party vehicle solutions (Ex: trains, planes, boats, motorcycles, etc).

5.1. Camera Player Input

This component is responsible for handling player input for all camera specific behaviors. You may now choose between the New Input System or the asset's Legacy Input System.

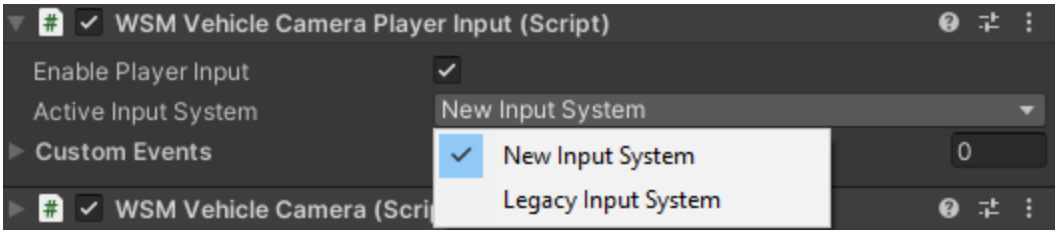


Figure: Active Camera Input System

5.1.1. Camera New Input System

The **new input system** leverages the power of the Input System Package to support a wide range of target platforms and controller devices (supported input devices).

The camera related input mappings are located on the "*WSM Vehicle Camera Input Mapping.inputactions*" file.

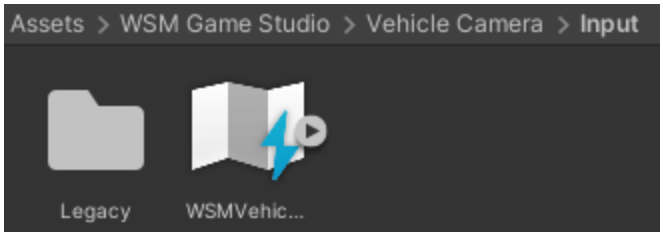


Figure: Camera Input Action Asset

By opening this file, you may expand and customize the input mappings as you see fit.

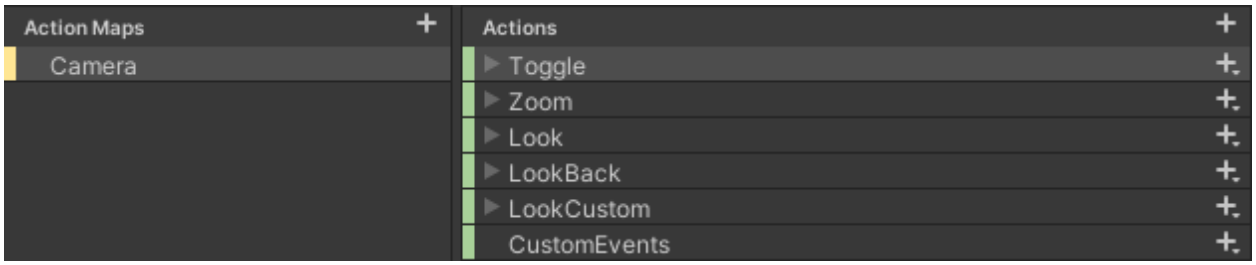


Figure: Camera Input Mappings

Sample mappings include control schemes for PC (**Keyboard**), Xbox and Playstation (**Gamepad**), and generic controllers (**USB Game Pad**).

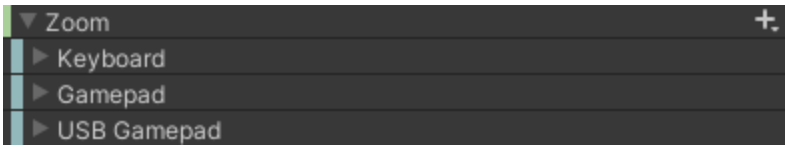


Figure: Input Mappings Sample

For example, on a **Xbox Controller**, camera zoom can be increased by holding **LB** and **D-pad Up.** (**L1** and **D-pad Up** on a **Playstation Controller**).

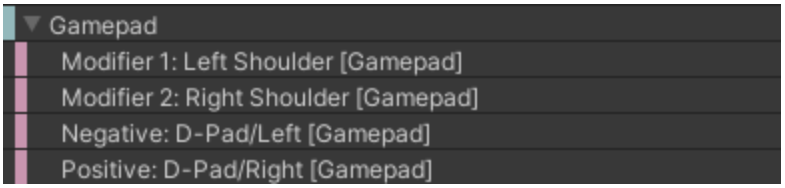


Figure: Console Input Sample (Xbox, Playstation)

5.1.2. Camera Legacy Input System

The **legacy input system** is an improved version of Unity's old Input Manager system (input mappings are stored on scriptable objects instead of the Project Settings).

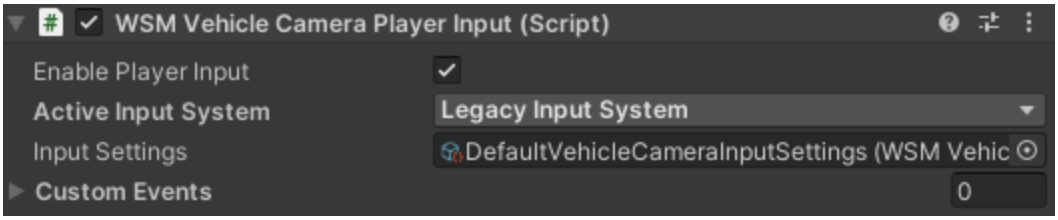


Figure: Camera Player Input

When the **legacy input system** is selected, the camera input mappings are stored on a Vehicle Camera Input Settings file.

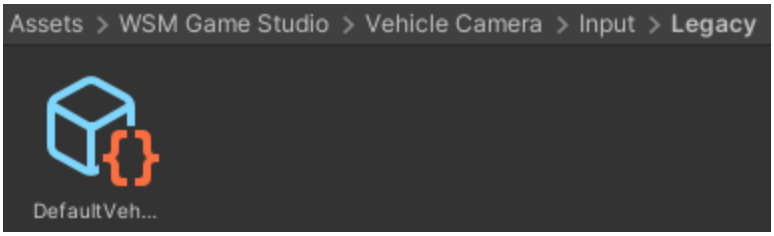


Figure: Camera Input Settings Location

By selecting this file, you can see and customize the camera input mappings.

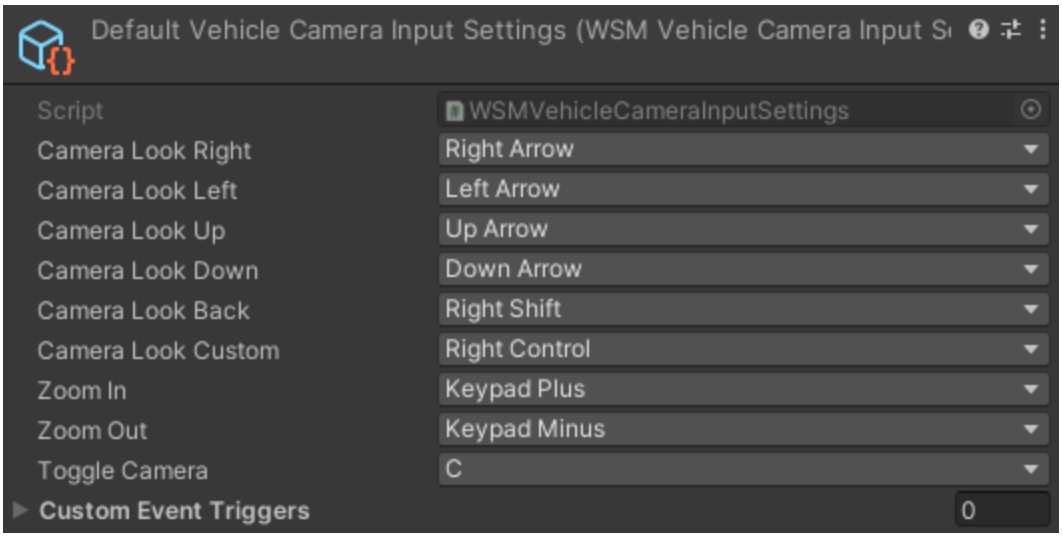


Figure: Camera Input Mappings

You can create custom input settings by right-clicking on your project and using Unity's Asset Creation menu: *Create* > *WSM Game Studio* > *Vehicle Camera* > *Legacy Input System* > *Vehicle Camera Input Settings*.

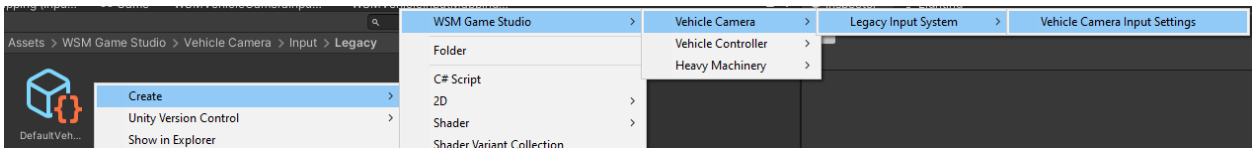


Figure: Creating a new Vehicle Camera Input Settings

You can create as many input settings as you wish, this can be very useful for testing different input mappings for your game.

5.2. Camera Component

The **WSM Vehicle Camera** component is responsible for controlling the vehicle camera.

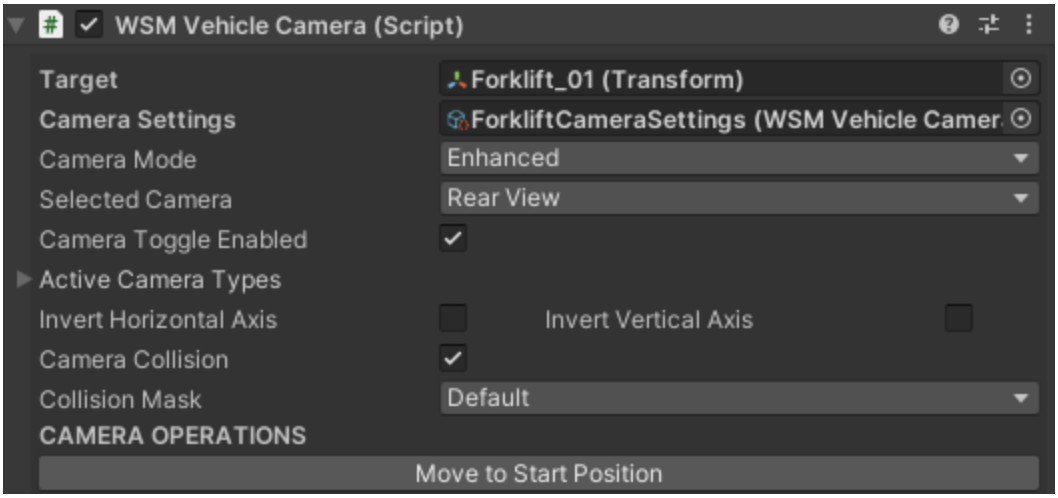


Figure: Vehicle Camera Component

5.2.1. Camera Mode

Starting with version v2.0, you can now select between **enhanced** and **legacy** camera modes.

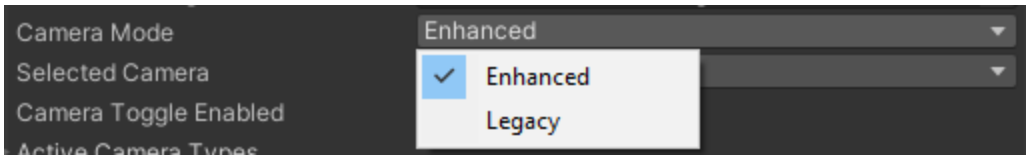


Figure: Selected Camera Mode

Enhanced Camera mode is based on modern game vehicle cameras. It supports 360º horizontal rotation, customizable vertical angle limit, camera damping for smooth following, camera collision and zoom.

Legacy Camera mode is based on older games from the PSX era. The camera is locked to fixed angles, allowing the player to look left, right, forward or backwards, while driving the vehicle (Camera damping and zoom support included on v2.0).

5.2.2. Target Vehicle & Target Settings

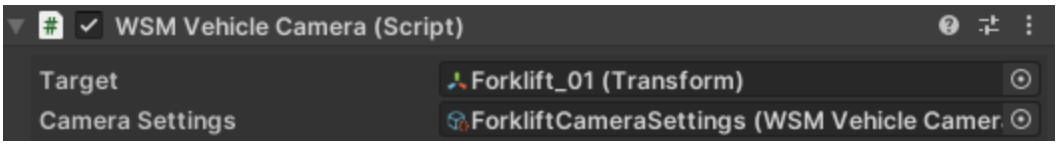


Figure: Target Vehicle & Target Settings

The camera tracks the **target** transform, regardless of the vehicle's type. **Camera settings** are stored as Scriptable Objects, allowing them to be shared by multiple vehicles or customized per vehicle type.

Note: For example, longer vehicles, like buses and trucks, usually need a greater zooming range to ensure the vehicle fits correctly on the screen.

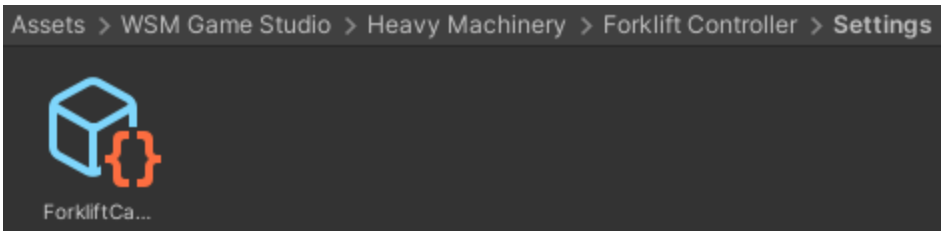


Figure: Camera Settings File

You can create custom camera settings by right-clicking on your project and using Unity's Asset Creation menu: *Create > WSM Game Studio > Vehicle Camera > Vehicle Camera Settings*.

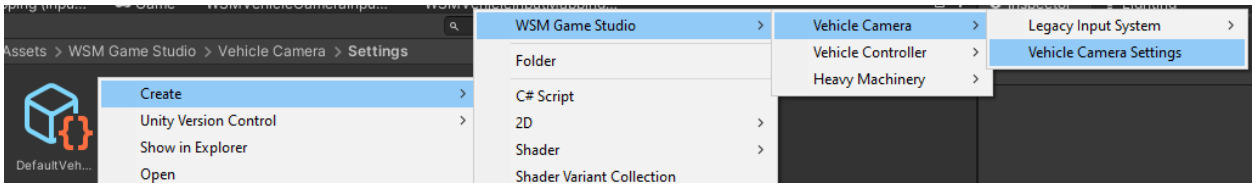


Figure: Creating a new Vehicle Camera Settings

The camera settings are divided per camera type to facilitate customization (more camera types may be included in the future). You can customize camera speed, zooming range, rotation limits, FOV distortion, damping speed, etc.

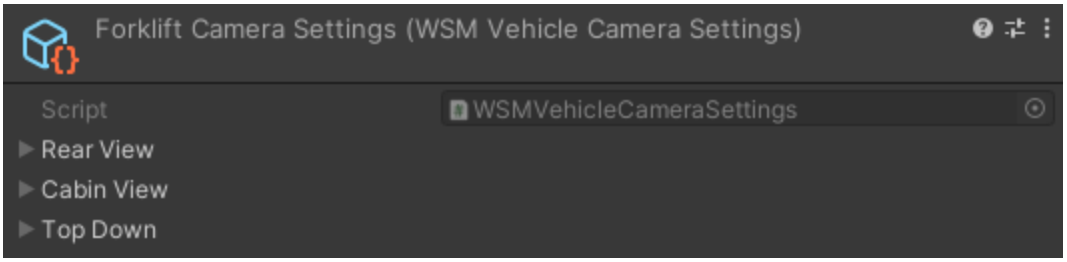


Figure: Vehicle Camera Settings

5.2.3. Supported Camera Types

Supported camera types are: **rear view**, **cabin view** and **top down**.

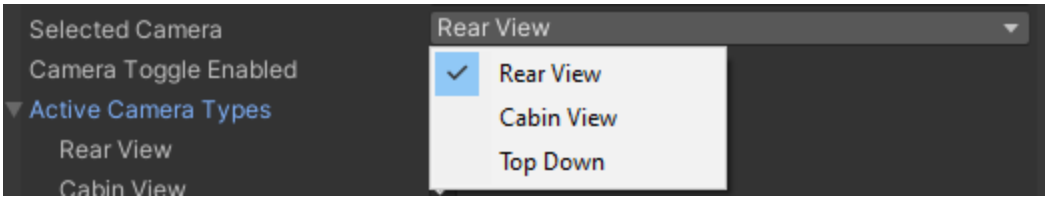


Figure: Selected Camera Type

In-game camera toggle is optional. You can also choose which camera types are available for in-game toggling.

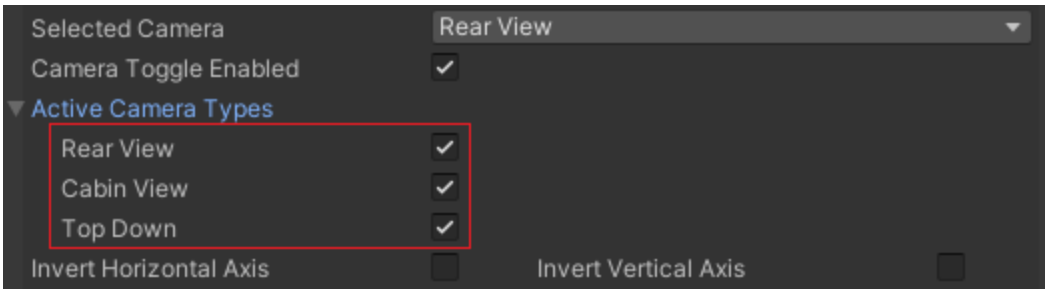


Figure: Active Camera Types

Prior to v2.0, the **Move to Start Position** button was used to apply the selected camera in-editor. It was kept for legacy support reasons.

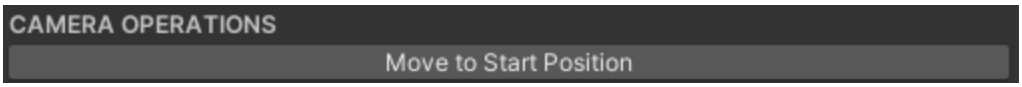


Figure: Move to Start Position

5.2.4. Camera Collision

Camera collision is enabled by default, but it is an optional feature. The **collision mask** is used to filter which layers may collide with the camera, giving you full control over what objects may block the camera.

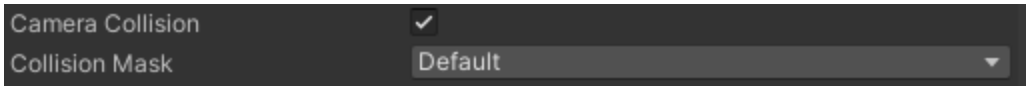


Figure: Camera Collision & Other Settings

6. Additional Info

6.1. Customization

This package includes a game-ready forklift model for sample purposes. More forklift models may be included in the future and/or sold separately as extensions (Here).

It is possible to create custom forklifts by using the sample prefabs as a starting point and replacing the sample models with custom or third-party models.

Just make sure your custom forklift main body, forks, primary mast, secondary mast and wheels meshes are separated from each other as shown in the sample image below.

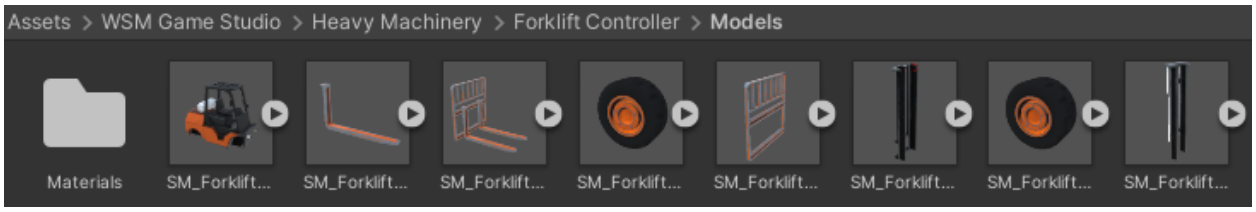


Figure: Forklift Part Meshes

6.2. Forklift Carrying Weight Capacity

As in real life, forklifts included in this asset have limited carrying weight capacity. Trying to raise cargo above this limit will shift the forklift's center of mass, making it harder to safely transport the cargo or, in case of really heavy cargo, to keep the forklift wheels on the ground.

The rule of thumb is, a forklift cannot carry cargo heavier than itself. For example, if you intend to lift a forklift, you need a bigger and heavier forklift to do the job.

For this asset, the recommended rigidbody mass value for cargo must not exceed half the

forklift mass. Since sample prefabs are set to a 1000 rigidbody mass, the recommended rigidbody mass value for cargo is between 22 and 500.

6.3. Rear Steering Vehicle Guidelines

The Vehicle Controller included in this package does support rear wheel steering vehicles.

Rear wheel steering vehicles, like forklifts for example, tend to easily oversteer on higher speeds which can result in undesired vehicle fishtailing.

To avoid fishtailing, increase the Steer Helper property value until your vehicle reaches the desired trade-off between realism and control.

Since vehicles are nonlinear systems, the optimal Steer Helper value will change according to your vehicle aerodynamics, center of mass and settings. It's recommended to test each vehicle and adjust the settings until the desired behavior is achieved.

7. License

This asset is commercialized through the Unity Asset Store and Fab.

For more information about Asset Store licensing, please refer to the Unity Asset Store EULA and FAQ. For more information about Fab licensing, please refer to Fab's EULA.

8. Contact Info & Support

If you have any questions, need support or have some business inquiries, feel free to get in touch.

Support requests are now being handled exclusively by email at wsmgamestudio@gmail.com

Your request must contain the following information:

- Asset Store Invoice Number
- Unity Editor Version
- Detailed description of the issue (including screenshots if possible)

The best way to reach me is by email at **wsmgamestudio@gmail.com**

https://wsmgamestudio.com/

Artstation

Unreal Marketplace

Unity Asset Store

Instagram

Discord

Youtube Channel

Facebook